AI CodeLens & Repo Chatter

Local-First AI Developer Productivity Platform

Document Type	Product Requirements Document (PRD)
Product Name	AI CodeLens & Repo Chatter
Version	1.0.0 — Initial Release
Status	DRAFT — For Review
Date	June 12, 2026
Target Audience	Engineering, Product, Design, Stakeholders
Deployment Model	Fully Local — Ollama + ChromaDB + React JS
Primary Model	qwen2.5-coder (via Ollama)

> *"Zero API cost. Zero cloud dependency. Infinite code insight."*

Table of Contents

1 Executive Summary

AI CodeLens & Repo Chatter is a **local-first, cost-free developer productivity platform** that delivers deep codebase intelligence without any cloud API dependencies. Engineered for high-velocity environments such as hackathons, intense sprint cycles, and resource-constrained teams, the product combines a LangChain ingestion backend, a ChromaDB vector store, and the open-weight **qwen2.5-coder** model running on Ollama to provide conversational access to any GitHub repository.

The system ingests repositories through language-aware AST-based chunking, indexes them locally, and answers natural language queries via a deterministic LCEL intent router. A cross-encoder reranker prunes retrieved context to fit the small model's context window, while the React split-pane dashboard streams responses and dynamically highlights the exact lines being explained — creating a seamless code-understanding loop that runs entirely on consumer hardware.

2 Problem Statement

2.1 Core Pain Points

- **Prohibitive API costs** — GPT-4/Claude API fees make always-on code assistants economically unviable for hackathons and indie developers.
- **Context-window blindness** — General chat models lack awareness of the full project structure; answers are generic and disconnected from the actual codebase.
- **Privacy & compliance risk** — Sending proprietary code to remote APIs violates NDA agreements and data-residency requirements.
- **Fragmented tooling** — Developers toggle between IDE, browser, docs, and chat windows; there is no unified pane for code-contextual Q&A.
- **Small-model reasoning limits** — Local open-weight models lack robust tool-calling and intent routing, making RAG pipelines unreliable without deterministic guardrails.

2.2 Opportunity

Consumer GPUs (RTX 3060+) now deliver sufficient VRAM to run 7–14B parameter code-specialized models at interactive speeds. Combined with local vector databases and deterministic routing logic, a fully offline, zero-cost, professional-grade code assistant is achievable today.

3 Product Vision & Goals

> **Vision:** Every developer — regardless of budget — should have a brilliant, privacy-preserving AI pair-programmer that truly understands their entire codebase.

3.1 Strategic Goals

- **G1 — Zero Operational Cost:** All inference and storage runs locally; no per-query API spend.
- **G2 — Deep Codebase Awareness:** AST-level chunking preserves semantic boundaries; retrieval is precise to function/class level.
- **G3 — Hackathon-Ready Speed:** Full repo ingestion and first query answered within 5 minutes on target hardware.

• **G4 — Privacy by Design:** No code, query, or response leaves the developer's machine.

• **G5 — Small-Model Reliability:** Deterministic LCEL routing eliminates prompt-engineering brittleness inherent to small models.

• **G6 — Intuitive UX:** Split-pane dashboard with live line-highlighting eliminates context-switching between chat and editor.

4 Target Users & Personas

Persona	Role	Primary Need	Key Pain
Hackathon Hacker	Solo / 2–3 person team	Rapid onboarding to unfamiliar repos under time pressure	No budget for API calls; needs answers in seconds
Enterprise Dev	Senior/Staff engineer	Navigate legacy monorepos without exposing code externally	IP compliance; slow context-loading in large codebases
Open-Source Contributor	OSS maintainer / contributor	Understand unfamiliar project conventions before first PR	Sparse docs; sprawling file trees
Code Reviewer	Tech lead / architect	Trace function call chains and dependency impacts quickly	Mental overhead of cross-file navigation
CS Student	Student / bootcamp learner	Learn from real production codebases interactively	No income for premium AI tools; needs guided exploration

5 System Architecture

The platform is composed of three tightly integrated layers: an **Ingestion Pipeline**, a **Retrieval & Inference Engine**, and an **Interactive React Dashboard**. All layers communicate over localhost; no external network calls are made at any point after the initial GitHub clone.

5.1 Ingestion Pipeline (Backend — Python / LangChain)

- **GitHub Clone** — GitPython clones the target repository into a sandboxed temp directory. Supports public repos and PAT-authenticated private repos.
- **Language Detection** — File extensions are mapped to AST splitter variants (Python → PythonCodeTextSplitter, JS/TS → RecursiveCharacterTextSplitter with language hint, etc.).
- **AST-Aware Chunking** — LangChain's language-specific splitters slice code at function and class boundaries, preserving logical completeness. Chunks carry file-path, start-line, end-line, and language metadata.
- **Embedding Generation** — Chunks are passed to the local Ollama embedding endpoint (nomic-embed-text) and stored in ChromaDB with metadata filters enabled.
- **Incremental Re-ingestion** — File modification timestamps are compared to ChromaDB records; only changed files are re-embedded, reducing re-index time by up to 80% on repeat runs.

5.2 Retrieval & Inference Engine (Backend — FastAPI / LangChain)

- **LCEL Intent Router** — Classifies incoming prompts into one of two deterministic pathways: (a) **File-Scoped Query** — user references a specific file path; retrieval is filtered to that document's chunks. (b) **Global Hybrid Search** — vector similarity + BM25 sparse retrieval across all indexed chunks.
- **Hybrid Search Fusion** — Reciprocal Rank Fusion (RRF) merges dense and sparse retrieval scores into a single ranked list, improving recall for identifier-heavy code queries.
- **Cross-Encoder Reranking** — A lightweight local cross-encoder (cross-encoder/ms-marco-MiniLM-L-6-v2) re-scores the top-K candidates and prunes to the final context window budget (≤4,096 tokens for qwen2.5-coder 7B).
- **Streaming Generation** — The pruned context + user query is sent to Ollama's streaming endpoint. Server-Sent Events (SSE) pipe tokens directly to the React frontend as they are produced.
- **Citation Injection** — The LLM is instructed via system prompt to annotate every code reference with a structured citation tag ([FILE:path:L{start}-L{end}]) that the frontend parser intercepts.

5.3 React Dashboard (Frontend)

- **Split-Pane Layout** — Left pane: file tree navigator + code viewer with syntax highlighting (Prism.js). Right pane: streaming chat interface.
- **Citation Parser** — SSE stream interceptor extracts citation tokens, triggers file-load if needed, and programmatically scrolls + highlights the referenced line range.
- **Ingestion Control Panel** — URL input + progress bar showing clone, chunk, embed, and index completion stages.
- **Model Health Monitor** — Polls Ollama's /api/tags endpoint; surfaces model load status and VRAM usage in the status bar.

6 Feature Specifications

F-01 — Repository Ingestion

- Accept GitHub HTTPS URL via dashboard input field.
- Display real-time ingestion progress: clone → detect → chunk → embed → index.
- Support repositories up to 500 MB and 50,000 files.
- Store ingestion metadata (repo URL, commit SHA, timestamp) in a local SQLite manifest.
- Allow re-ingestion with incremental update (delta only).

F-02 — AST-Aware Code Chunking

- Support Python, JavaScript, TypeScript, Java, Go, Ruby, C/C++, Rust.
- Chunk at function/method/class declaration boundaries.
- Preserve shebang lines, decorators, and docstrings within the owning chunk.
- Fall back to recursive character splitter for unsupported file types.
- Chunk size target: 512 tokens; overlap: 64 tokens.

F-03 — Hybrid Semantic Search

- Dense retrieval via ChromaDB cosine similarity (nomic-embed-text embeddings).
- Sparse retrieval via BM25 index built over chunk text.
- RRF fusion of dense + sparse ranked lists.
- Metadata filter support: language, file path prefix, last modified date.
- Configurable top-K (default: 20 pre-rerank, 5 post-rerank).

F-04 — Deterministic LCEL Intent Router

- Two routing branches: File-Scoped and Global Search.
- File-Scoped trigger: prompt contains a resolvable relative file path present in the index.
- Global Search: default for all other prompts.
- Routing decision logged for debugging; surfaced as a subtle UI badge.
- No LLM call required for routing; regex + path resolution only.

F-05 — Cross-Encoder Reranking

- Local cross-encoder model: ms-marco-MiniLM-L-6-v2 (22 MB, CPU-compatible).
- Reranks top-K retrieved chunks against the full user query.
- Hard token budget enforcement: sum of retained chunks ≤ context_window - 512 (reserved for response).
- Chunk truncation at boundary (never mid-token) if budget is tight.

F-06 — Streaming Chat Interface

- SSE-based streaming from FastAPI backend to React client.
- Token-by-token render with blinking cursor.
- Citation tokens rendered as inline clickable chips.
- Stop generation button; response copy-to-clipboard.

- Persistent session history (localStorage) with search/filter.

F-07 — Citation-Driven Code Highlighting

- Parser intercepts [FILE:path:Lstart-Lend] tokens mid-stream.
- Lazy-loads file content from backend /file endpoint if not already open.
- Scrolls code pane to the cited line range.
- Highlights lines with animated yellow fade-in; fades out after 3 seconds.
- Multiple citations in one response each trigger sequential highlights.

F-08 — Local Model Health Dashboard

- Sidebar widget showing: active Ollama models, VRAM usage, inference latency (last 5 queries).
- One-click model pull for qwen2.5-coder variants (7B, 14B).
- Warning banner if available VRAM < recommended minimum for selected model.

7 Technical Stack

Layer	Technology	Version	Rationale
LLM Runtime	Ollama	0.3+	Local model serving; OpenAI-compatible API; GPU offload
Code LLM	qwen2.5-coder	7B / 14B	SOTA local code model; low VRAM footprint; instruction-tuned
Embeddings	nomic-embed-text	1.5	137M param; 8,192 context; free via Ollama
Vector Store	ChromaDB	0.5+	Embedded mode; persistent SQLite backend; metadata filters
RAG Framework	LangChain (LCEL)	0.3+	Composable chains; LCEL enables deterministic routing
Reranker	sentence-transformers cross-encoder	3.0+	CPU-runnable; 22 MB; MiniLM-L-6 precision
API Server	FastAPI + Uvicorn	0.115+	Async SSE streaming; OpenAPI docs; pydantic validation
Frontend Framework	React + Vite	18 / 5+	Fast HMR; component-based split-pane architecture
Code Highlighting	Prism.js	1.29+	150+ languages; lightweight; line-number plugin
Sparse Index	rank_bm25	0.2+	Pure-Python BM25+; no external service required
Repo Cloning	GitPython	3.1+	Programmatic git clone; PAT auth support
OS / Runtime	macOS / Ubuntu / Windows WSL2 + Python 3.11	—	Cross-platform; CUDA/Metal GPU acceleration optional

8 Data Flow & Pipeline Design

8.1 Ingestion Flow

Actor	Action
User	Enters GitHub URL in dashboard → clicks Ingest
Frontend	POST /api/ingest {repo_url}
Backend	GitPython.clone_from() → temp directory
Backend	Walk directory tree; classify files by extension
Backend	Apply language-specific AST splitter per file
Backend	Batch chunks (64 per batch) → Ollama embed endpoint
Backend	Upsert vectors + metadata → ChromaDB collection
Backend	Write ingestion manifest → SQLite
Frontend	SSE progress stream renders completion per stage

8.2 Query Flow

Actor	Action
User	Types natural language query in chat input
Frontend	POST /api/chat {query, session_id, open_file?}
LCEL Router	Regex checks for file path pattern in query
LCEL Router	Branch A (file-scoped): filter ChromaDB to file chunks → top-20
LCEL Router	Branch B (global hybrid): BM25 + dense → RRF merge → top-20
Reranker	Cross-encoder scores all 20 candidates against query
Reranker	Sort descending; truncate to token budget (≤4,096 - 512)
Ollama	Streaming inference with system prompt + pruned context + query
Backend	SSE stream tokens + citation markers to frontend
Frontend	Render tokens; parse citations; highlight code pane

9 Non-Functional Requirements

Category	Requirement	Target
Performance	Time-to-first-token (TTFT) for chat responses	≤ 3 seconds on RTX 3060 12 GB
	Full repo ingestion for 10k-file repo	≤ 5 minutes on target hardware
	Reranker latency (top-20 candidates)	≤ 200 ms on CPU
Scalability	Max repo size supported	500 MB / 50,000 files
Scalability	Concurrent user sessions (local dev server)	1–5 (developer-local tool)
Reliability	Backend process uptime during active session	99% (crash recovery via PM2 optional)
Security	Data exfiltration surface	Zero — no external network calls post-clone
Security	Authentication for local server	Optional API key header for multi-user LAN deployments
Usability	Setup time from zero to first query (experienced dev)	≤ 15 minutes
Usability	UI responsiveness during streaming	No janky scroll; smooth token append at ≥ 60 fps
Compatibility	Operating systems	macOS 13+, Ubuntu 22.04+, Windows 11 WSL2
Compatibility	GPU backends	NVIDIA CUDA 12+, Apple Silicon Metal, CPU fallback
Maintainability	Code coverage target (backend)	≥ 70% unit tests via pytest

10 Feature Prioritization

Priorities follow the MoSCoW-derived P0–P3 scale. **P0** = must-ship for MVP launch. **P1** = high-value; ship in Sprint 2. **P2** = important; ship in Sprint 3. **P3** = future / nice-to-have.

Feature	Priority	Effort	Sprint	Notes
Repository Ingestion (clone + index)	P0	M	S1	Core pipeline; no product without it
AST-Aware Chunking (8 languages)	P0	M	S1	Quality of retrieval depends on chunk quality
ChromaDB vector store + metadata filters	P0	S	S1	Persistence layer for embeddings
LCEL Intent Router (2 branches)	P0	S	S1	Reliability; avoids small-model tool-call flakiness
Streaming chat via SSE	P0	S	S1	Core UX; blocking response is unacceptable
Cross-Encoder Reranking	P0	S	S1	Context budget management for 7B model
Citation-Driven Code Highlighting	P0	M	S1	Key differentiator; defines the product experience
Split-Pane React Dashboard	P0	L	S1	Primary UI shell
Incremental Re-ingestion	P1	M	S2	Critical for hackathon velocity
BM25 Hybrid Search Fusion (RRF)	P1	S	S2	Improves recall for identifier queries
Model Health Monitor widget	P1	S	S2	Developer ergonomics; GPU awareness
Persistent chat history (localStorage)	P1	S	S2	Session continuity
Multi-repo support (switch between indices)	P2	M	S3	Power-user workflow
Private repo support (PAT auth)	P2	S	S3	Enterprise use case
qwen2.5-coder 14B auto-selection if VRAM ≥ 20G	P2	S	S3	Quality uplift on capable hardware
VS Code extension (open-from-IDE)	P3	XL	S4	Stretch goal; deepens IDE integration
Multi-turn conversation memory (RAG + chat hist)	P3	L	S4	Complex; requires careful token budgeting
Team / LAN mode (shared ChromaDB server)	P3	XL	S4	Changes security model significantly

11 Risk Register

Risk	Likelihood	Impact	Mitigation
qwen2.5-coder VRAM overflow on 8 GB GPUs	High	High	Detect VRAM at startup; auto-select 7B Q4_K_M quant; surface warning if < 6 GB available
AST splitter failure on minified / transpiled JS	High	Medium	Fallback to RecursiveCharacterTextSplitter with 512-char chunks; log warning in UI
ChromaDB corruption on unclean shutdown	Medium	High	Write-ahead log enabled; provide one-click 're-index' button; backup on ingest completion
Context window overflow (large files, deep call stacks)	High	Medium	Hard token budget in reranker; truncate at chunk boundary; test with 10k-LOC files
Ollama API breaking changes	Low	High	Pin Ollama version in install script; abstract behind OllamaClient wrapper class
Slow embedding on CPU-only hardware	Medium	Medium	Batch size auto-tuning; progress bar with ETA; document minimum hardware specs clearly
Citation regex false-positives corrupting code pane	Medium	Medium	Strict citation pattern with file-existence validation before highlight trigger
Large monorepo clone timeout (>500 MB)	Medium	Low	Shallow clone (depth=1) by default; configurable depth; repo size check before clone
LangChain deprecations (LCEL API changes)	Low	Medium	Pin LangChain version; integration test suite covers router branches

12 Milestones & Roadmap

M1 — Sprint 1 — MVP (Weeks 1–3)

- Backend: repo clone, AST chunking, embed pipeline, ChromaDB indexing
- Backend: LCEL router (2 branches), cross-encoder reranker, SSE streaming endpoint
- Frontend: split-pane shell, ingestion progress UI, basic chat with streaming
- Frontend: citation parser + code highlighting (single citation per response)
- Dev: Docker Compose for backend stack; Vite dev server for frontend
- Deliverable: Demo-ready prototype; can answer questions about a real GitHub repo

M2 — Sprint 2 — Quality & Ergonomics (Weeks 4–6)

- Incremental re-ingestion (delta-only re-embed)
- BM25 hybrid search + RRF fusion integrated
- Model health monitor sidebar widget
- Persistent chat session history
- Multiple citation highlights per response
- Deliverable: Beta release to internal testers; performance profiling complete

M3 — Sprint 3 — Enterprise-Ready (Weeks 7–9)

- Private repo support via GitHub PAT
- Multi-repo index switching
- 14B model auto-selection based on VRAM detection
- Unit test coverage ≥ 70%; integration test suite for router branches
- Installation script (Homebrew formula + apt PPA)
- Deliverable: v1.0.0 public release; README + docs site

M4 — Sprint 4 — Ecosystem (Weeks 10–14)

- VS Code extension (open file from IDE → highlight in CodeLens)
- Multi-turn conversation memory with token-budgeted history injection
- Team / LAN mode with shared ChromaDB server
- Deliverable: v1.1.0 feature release

13 Success Metrics

Metric	Target	Measurement Method
Time-to-First-Token (TTFT)	≤ 3 s (RTX 3060 12 GB)	Automated latency test suite; 50 queries benchmark set
Retrieval Precision@5	≥ 0.70	Manual eval set of 50 Q&A; pairs over 3 test repos
Citation Accuracy	≥ 90% correct file+line references	Automated parser unit tests + manual spot-check
Ingestion Throughput (10k files)	≤ 5 minutes	Timed run on reference hardware (RTX 3060 + Ryzen 7)
Reranker Latency	≤ 200 ms (top-20 candidates, CPU)	pytest-benchmark across 100 random query/chunk pairs
GitHub Stars (3 months post-launch)	≥ 500	GitHub repository analytics
User-Reported Setup Success Rate	≥ 85% first-run success	Discord/issue tracker feedback; setup error telemetry (opt-in)

14 Constraints & Assumptions

14.1 Constraints

- **Hardware floor:** Minimum 8 GB GPU VRAM (16 GB recommended for 14B model).
- **Internet required for initial clone only** — subsequent queries are fully offline.
- **Single-user by default** — LAN/team mode is a Sprint 4 feature.
- **No real-time file-watch** — users must manually trigger re-ingestion after significant code changes.
- **Context window ceiling** — qwen2.5-coder 7B supports 32k tokens; pipeline budget-caps to 4k for stability.
- **No IDE integration in MVP** — standalone web app only until VS Code extension (Sprint 4).

14.2 Assumptions

- Users have Python 3.11+, Node 18+, and Git installed on their system.
- Ollama is installed and ollama pull qwen2.5-coder has been run before first launch.
- Target GitHub repositories are primarily code (not binary-heavy asset repos).
- The majority of target files are in the 8 supported languages; other files are indexed as plain text.
- Users accept that response quality is lower than GPT-4-level cloud models in exchange for zero cost and privacy.
- No authentication system is required for MVP (single developer, localhost only).

15 Open Questions

ID	Question
OQ-01	Should we support embedding model selection (e.g. mxbai-embed vs nomic-embed-text) in v1.0, or standardize on nomic-embed-text to reduce configuration surface?
OQ-02	What is the optimal BM25 weight vs dense weight in the RRF fusion? Needs ablation study on at least 3 diverse repos.

OQ-03	Should ChromaDB collections be scoped per repo (separate collection per clone) or a single collection with repo-id metadata filter? Impacts query isolation and deletion logic.
OQ-04	How should we handle binary files (images, compiled artifacts) — skip silently, index filename only, or surface as a UI warning?
OQ-05	Is a Docker-first installation (docker-compose up) preferable over native scripts for the target hackathon audience? Tradeoff: Docker adds GPU passthrough complexity.
OQ-06	Should the citation format ([FILE:path:Lstart-Lend]) be user-visible or invisibly consumed by the frontend? Current design shows chips — is that preferred?
OQ-07	What telemetry, if any, should be collected opt-in? Candidates: query latency, retrieval precision self-report, error types.

AI CodeLens & Repo Chatter — PRD v1.0.0 — DRAFT — June 12, 2026 — All components run entirely on local consumer hardware. No data leaves the developer's machine.